

Prolog Programming - Basics

Dr. Nirmala Shinde Baloorkar
Assistant Professor

Department of Computer Engineering, KJSSE

Outline

- What is Prolog?
- Prolog Building Blocks
- How to Execute Prolog?

What is Prolog?

- Prolog = Programming in Logic
- Used in Artificial Intelligence and Expert Systems
- Prolog programs describe **facts and rules**
- The system find answers automatically

Prolog Building Blocks

1. Facts
2. Rules
3. Queris

Prolog Basic - Fact

Facts

- A fact represents a relationship or property between the objects.

```
relation(object1,object2...).
```

- Example

1. Tom is a cat.
2. Kunal loves to eat Pasta.
3. Hair is black.

Prolog Basic – Facts (continued)

Following are some guidelines to write facts –

- Names of properties/relationships begin with lower case letters.
- Objects appear as comma-separated arguments within parentheses.
- A period "." must end a fact.

```
relation(object1,object2...).
```

1. cat(tom).
2. loves_to_eat(kunal,pasta).
3. of_color(hair,black).

Execution and Queries on Fact

<https://swish.swi-prolog.org>

**SWISH** File Edit
Program x +
1 `cat(tom).`
2 `loves_to_eat(kunal, pasta).`
3 `of_color(hair, black).`

```
?- cat(tomy).
```


 `cat(tomy).`
false

```
?- loves_to_eat(kunal, X).
```


 `loves_to_eat(kunal, X).`
X = pasta

```
?- of_color(X,Y).
```


 `of_color(X,Y).`
X = hair,
Y = black

Prolog Basic - Rules

Rules

- It describes conditional relationship.
- Suppose we have some rules as given below –
 1. Lili is happy if she dances.
 2. Tom is hungry if he is searching for food.
 3. Jack and Lili are friends if both of them love to play cricket.

```
rule_name(object1, object2, ...) :- fact/rule(object1,  
object2, ...)
```


Prolog Basic – Rules (continued)

```
rule_name(object1, object2, ...) :- fact/rule(object1,  
object2, ...)
```

- Suppose we have some rules as given below –
 1. Lili is happy if she dances.
happy(lili) :- dances(lili).
 2. Tom is hungry if he is searching for food.
hungry(tom) :- search_for_food(tom).
 3. Jack and Lili are friends if both of them love to play cricket.
friends(jack, bili) :- lovesCricket(jack), lovesCricket(bili).

Prolog Basic – Knowledge Base

- Knowledge base = database
 - A Knowledge Base is a collection of facts and rules.

female(lili).

dances(lili).

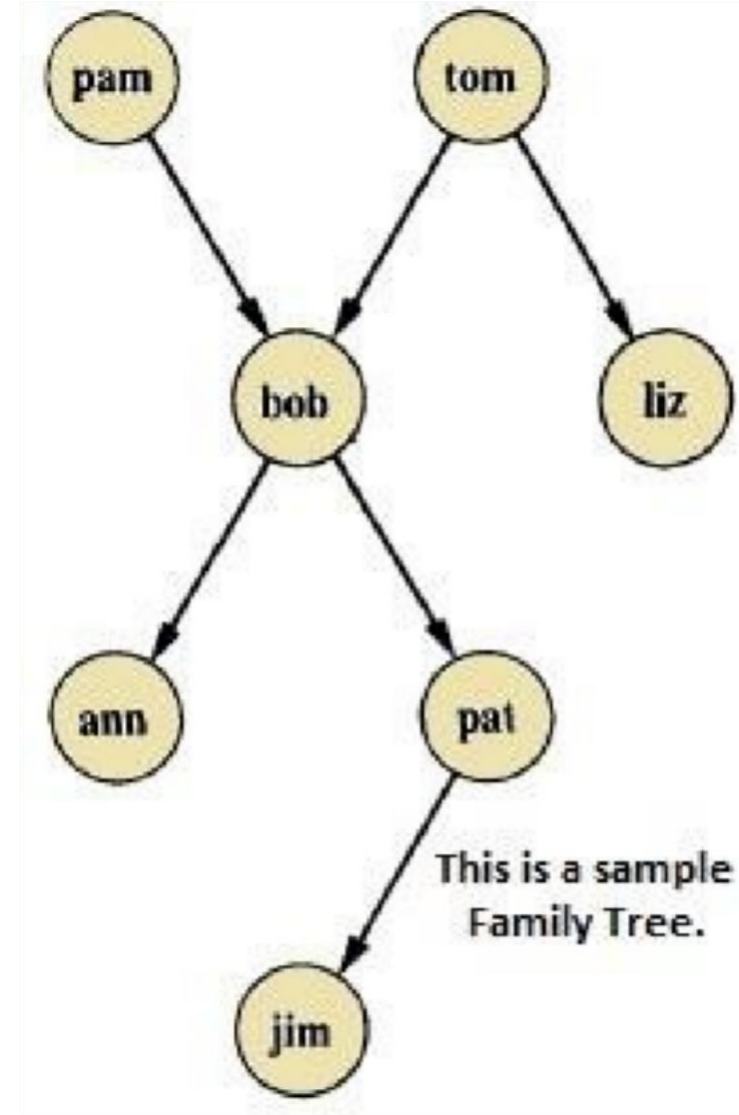
happy(lili) :- dances(lili).

Family Tree

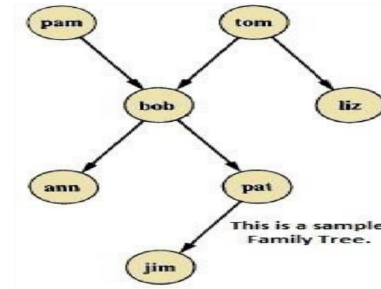
parent(pam, bob).
parent(tom, bob).
parent(tom, liz).
parent(bob, ann).
parent(bob, pat).
parent(pat, jim).

female(pam).
female(liz).
female(pat).
female(ann).

male(jim).
male(tom).
male(bob).



Family Tree - Example



parent(pam, bob).

parent(tom, bob).

parent(tom, liz).

parent(bob, ann).

parent(bob, pat).

parent(pat, jim).

female(pam).

female(liz).

female(pat).

female(ann).

male(jim).

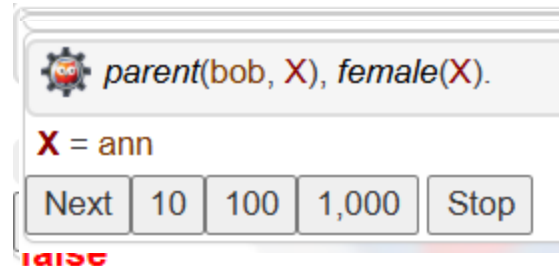
male(tom).

male(bob).

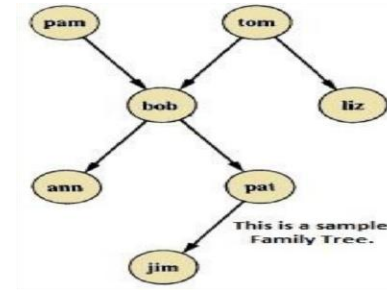
Find all children of “bob” who are female.

Query with Multiple Conditions

```
?- parent(bob, X), female(X).
```



Family Tree - Example



parent(pam, bob).

parent(tom, bob).

parent(tom, liz).

parent(bob, ann).

parent(bob, pat).

parent(pat, jim).

female(pam).

female(liz).

female(pat).

female(ann).

male(jim).

male(tom).

male(bob).

Find Bob's children who are NOT female.

Query with Multiple Conditions

?- parent(bob, X), \+ female(X).

parent(bob, X), \+ female(X).

false

?- parent(tom, X), \+ female(X).

parent(tom, X), \+ female(X).

X = bob

Next

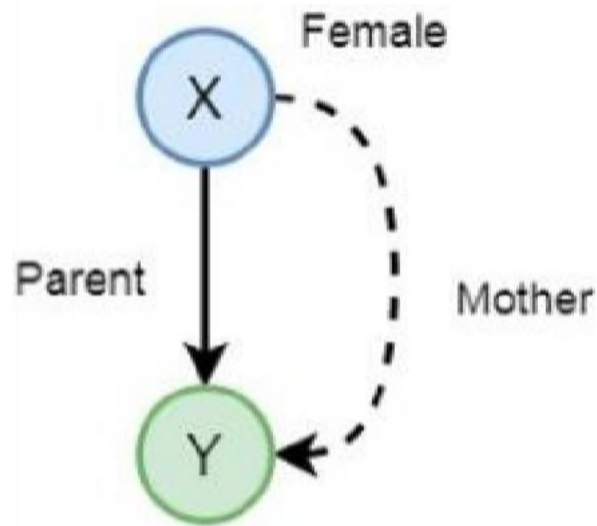
10

100

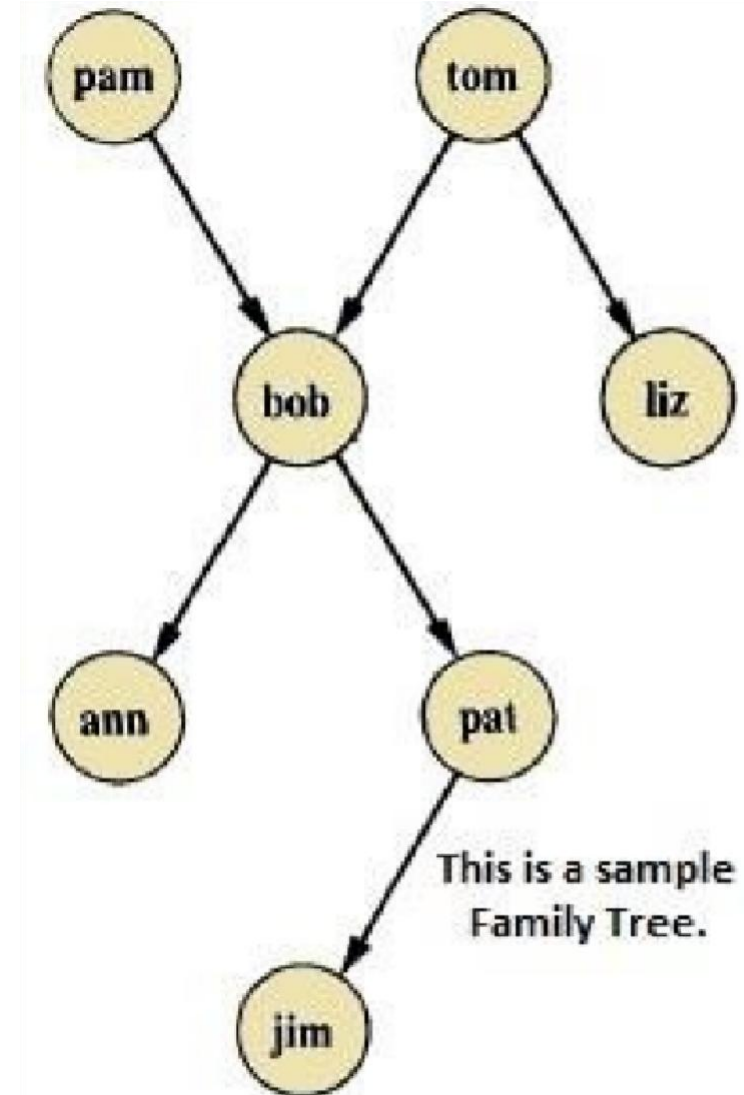
1,000

Stop

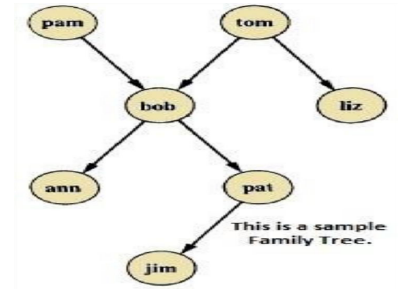
Family Tree – Mother Relation



$\text{mother}(X, Y) \text{ :- } \text{parent}(X, Y), \text{female}(X).$



Family Tree – Mother Relation



```
1 parent(pam, bob).
2 parent(tom, bob).
3 parent(tom, liz).
4 parent(bob, ann).
5 parent(bob, pat).
6 parent(pat, jim).
7 parent(peter, jim).
8
9 female(pam).
10 female(liz).
11 female(pat).
12 female(ann).
13
14 male(tom).
15 male(bob).
16 male(jim).
17
18 mother(X,Y) :- parent(X,Y), female(X).
19
```

```
?- mother(pam, Y).
```

```
mother(pam, Y).
```

Y = bob

```
?- mother(X, Y).
```

```
mother(X, Y).
```

X = pam,

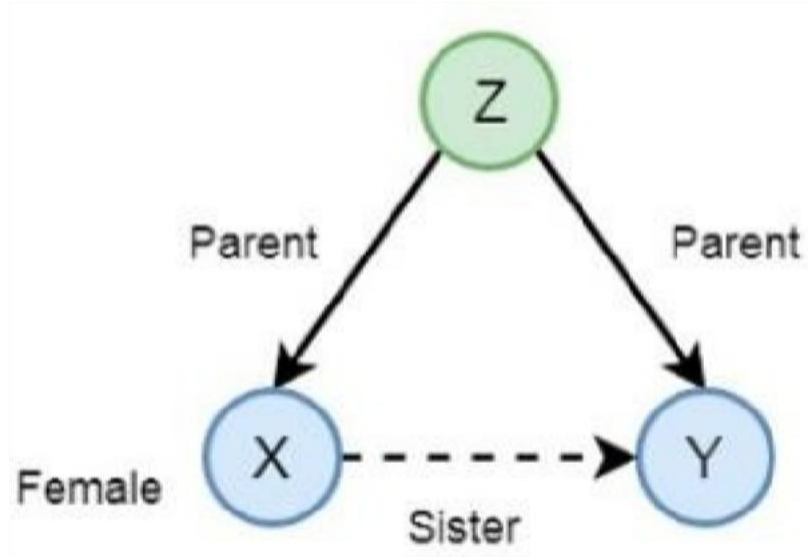
Y = bob

X = pat,

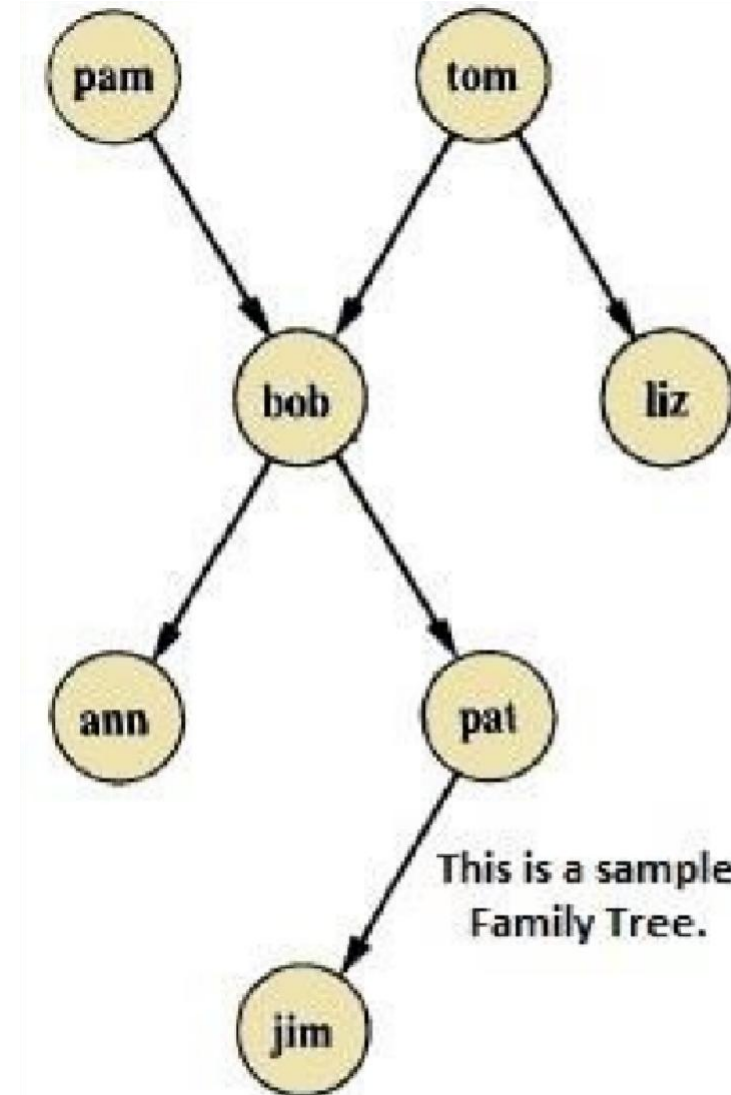
Y = jim

false

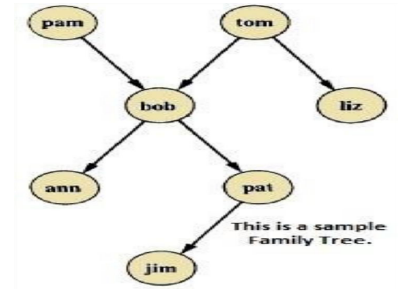
Family Tree - Sister Relation



$\text{sister}(X, Y) :- \text{parent}(Z, X), \text{parent}(Z, Y),$
 $\text{female}(X), X \neq Y.$



Family Tree – Mother Relation



```
1 parent(pam, bob).
2 parent(tom, bob).
3 parent(tom, liz).
4 parent(bob, ann).
5 parent(bob, pat).
6 parent(pat, jim).
7 parent(peter, jim).
8
9 female(pam).
10 female(liz).
11 female(pat).
12 female(ann).
13
14 male(tom).
15 male(bob).
16 male(jim).
17
18 mother(X,Y) :- parent(X,Y), female(X).
19 sister(X, Y) :- parent(Z, X), parent(Z, Y), female(X), X \== Y.
```

?- sister(ann, pat).

 sister(ann, pat).

true

Rule base Vs knowledgebase programming (C/Java Vs Prolog/LISP)

	Rule Based System	Knowledge Based System
Can Process	Data Rules	Data Rules Knowledge
Can Output	Information Decision	Information, Decision, Expert Advice, Recommendations
Commonly used for	Enterprise rules	Domain specific rules
Ideal for	Simplistic business rules	Complex business rules
Best for these types of applications	Decision Compliance	Advising, Product Selection, Recommending, Troubleshooting
Domain	Broad Logic	Deep Logic

Queries?

Thank you!!